

Real-Time Email Scheduler â Project Documentation

This project is a full-stack â send now / schedule laterâ email system built around a **queue + worker** architecture, with a **real-time dashboard** showing job status.

1) What you built (in 1 minute)

You have three main moving parts:

1. **Frontend (React/Vite)**: UI to compose emails and view job history/status.
2. **API server (Node/Express + Socket.IO)**: REST endpoints to create jobs and read job history, and a WebSocket server to broadcast job lifecycle events.
3. **Worker (BullMQ Worker)**: background process that actually sends the email when BullMQ releases the job (immediately or after a delay).

Supporting services:

- â€¢ **Redis**: stores the BullMQ queue + delayed jobs.
- â€¢ **PostgreSQL**: stores durable job records and job event timeline.
- â€¢ **MailHog (dev)**: captures outgoing emails so you can view them in a browser.

2) Repository structure

Top-level folders:

- â€¢ 'frontend/' â React + Vite UI
- â€¢ 'backend/' â Docker Compose + Node backend code ('backend/app/')

Backend code layout ('backend/app/')

- â€¢ 'server.js' â Express API + Socket.IO server
- â€¢ 'worker.js' â BullMQ worker that processes queued jobs
- â€¢ 'routes/' â HTTP routes
- â€¢ 'controllers/' â route handlers (enqueue jobs, query jobs, retry/delete)
- â€¢ 'jobs/' â BullMQ queue configuration + processors
- â€¢ 'services/' â email sending logic
- â€¢ 'utils/' â Redis + Nodemailer transport
- â€¢ 'db/' â Knex connection + migrations

Frontend layout ('frontend/src/')

- â€¢ 'main.jsx' / 'App.jsx' â app bootstrap and routes
- â€¢ 'pages/ComposePage.jsx' â Compose Email page
- â€¢ 'components/Dashboard.jsx' â dashboard view (table + job details)
- â€¢ 'hooks/useSocket.js' â Socket.IO client connection

â€¢ 'hooks/useJobs.js' â€¢ fetch jobs + refresh on socket events
â€¢ 'services/emailApi.js' / 'services/jobsApi.js' â€¢ Axios calls to backend

3) How to run locally (recommended)

3.1 Start backend services (Docker Compose)

From 'backend/':

```
docker compose up --build
```

This starts:

â€¢ API server: 'http://localhost:3000'
â€¢ Redis: 'localhost:6379'
â€¢ Postgres: 'localhost:5432'
â€¢ Worker: connects to Redis + API WebSocket
â€¢ MailHog UI: 'http://localhost:8025' (SMTP on 'localhost:1025')

3.2 Configure and start the frontend

The frontend expects an environment variable:

â€¢ 'VITE_API_BASE' â€¢ base URL for **both** Axios and Socket.IO client

Create 'frontend/.env':

```
VITE_API_BASE=http://localhost:3000
```

Then from 'frontend/':

```
npm install  
npm run dev
```

Vite prints the local dev URL (usually 'http://localhost:5173').

4) Main workflows (end-to-end)

Workflow A â€¢ Send nowâ€¢

1. User fills the â€¢ Compose Emailâ€¢ form and chooses **Send now**.
2. Frontend calls 'POST /api/send-email'.
3. API enqueues a BullMQ job in Redis (attempts + exponential backoff).
4. API inserts a row into Postgres table 'jobs' and a 'job_events' entry ('queued').

5. API broadcasts 'job:queued' via Socket.IO.
6. Worker receives job from Redis and marks it **active** in Postgres + inserts 'started' event.
7. Worker sends the email via Nodemailer (MailHog in dev).
8. Worker marks job **completed** (or **failed**) and writes a 'completed'/'failed' event.
9. Worker emits 'job:started', 'job:completed', or 'job:failed' so the dashboard refreshes live.

Workflow B → Schedule for later

1. User chooses **Schedule** and selects a local datetime.
2. Frontend converts that datetime to **UTC ISO string** ('toISOString').
3. Frontend calls 'POST /api/schedule-email' with 'sendAt'.
4. API computes 'delay = sendAt - now' and creates a delayed BullMQ job.
5. The job sits in Redis until it is due, then the worker processes it the same way as → Send now .

Workflow C → Retry failed job

1. Dashboard calls 'POST /api/jobs/:id/retry' for a job whose status is 'failed'.
2. API creates a **new** BullMQ job with the same email payload.
3. API inserts a new 'jobs' row and records a 'requeued' event.

Workflow D → Delete job

1. Dashboard calls 'DELETE /api/jobs/:id'.
2. API tries to remove the job from BullMQ (if still present).
3. API deletes the job and its events from Postgres.

5) Backend API reference

Base URL: 'http://localhost:3000'

Send immediately

⌘ 'POST /api/send-email'
 ⌘ Body:
 ⌘ 'to' (string)
 ⌘ 'subject' (string)
 ⌘ 'body' (string)
 ⌘ Response: '{ queued: true, jobId: "..." }'

Schedule for later

⌘ 'POST /api/schedule-email'
 ⌘ Body:
 ⌘ 'to' (string)
 ⌘ 'subject' (string)

â€¢ 'body' (string)
â€¢ 'sendAt' (string, ISO datetime in the future; recommended UTC via 'toISOString()')
â€¢ Response: '{ scheduled: true, jobId: "..." }'

List jobs (dashboard table)

â€¢ 'GET /api/jobs'
â€¢ Response: array of job rows from Postgres (most recent first)

Job details + timeline (modal)

â€¢ 'GET /api/jobs/:id'
â€¢ Response:
â€¢ 'job' â€¢ row from 'jobs'
â€¢ 'events' â€¢ ordered rows from 'job_events'

Stats (not currently used by UI)

â€¢ 'GET /api/stats'
â€¢ Response: grouped counts by 'status'

Retry failed job

â€¢ 'POST /api/jobs/:id/retry'
â€¢ Notes: only allowed when 'status === "failed"'

Delete job

â€¢ 'DELETE /api/jobs/:id'

6) Real-time events (Socket.IO)

The API server runs Socket.IO on the same host/port as the REST API ('http://localhost:3000').

Frontend listens for:

â€¢ 'job:queued'
â€¢ 'job:started'
â€¢ 'job:completed'
â€¢ 'job:failed'

When any of these are received, the frontend re-fetches 'GET /api/jobs' to refresh the table.

7) Database schema (PostgreSQL via Knex migrations)

'jobs' table ('backend/app/db/migrations/001_create_jobs.js')

- â€¢ 'id' (string, primary key) â€¢ uses BullMQ job id
- â€¢ 'type' (string) â€¢ 'send' or 'scheduled'
- â€¢ 'to_email', 'subject', 'body'
- â€¢ 'status' â€¢ 'queued' | 'active' | 'completed' | 'failed'
- â€¢ 'attempts' (int), 'failed_reason' (text)
- â€¢ 'queued_at', 'started_at', 'completed_at' (timestamps)

'job_events' table ('backend/app/db/migrations/002_create_job_events.js')

- â€¢ 'id' (auto increment)
- â€¢ 'job_id' (string)
- â€¢ 'event_type' â€¢ 'queued', 'started', 'completed', 'failed', 'requeued'
- â€¢ 'data' (jsonb) â€¢ optional metadata (error, sendAt, etc.)
- â€¢ 'created_at' (timestamp)

8) Background processing (BullMQ)

Queue:

- â€¢ 'backend/app/jobs/emailQueue.js' creates a BullMQ queue named 'emailQueue'.

Worker:

- â€¢ 'backend/app/worker.js' starts a BullMQ 'Worker("emailQueue", ...)'.
 - â€¢ It listens to 'active', 'completed', 'failed' events and:
 - â€¢ updates 'jobs' table
 - â€¢ inserts 'job_events'
 - â€¢ emits Socket.IO events (via a Socket.IO client connected to the API server)

Processor:

- â€¢ 'backend/app/jobs/processors/sendEmail.js' calls 'emailService.send(...)'.
 - â€¢

9) Email delivery in development (MailHog)

In Docker Compose, SMTP is set to:

- â€¢ Host: 'mailhog'
- â€¢ Port: '1025'

MailHog inbox UI:

- â€¢ 'http://localhost:8025'

So when the worker sends an email, youâ€™ll see it in MailHog instead of a real inbox.

10) Notes / gotchas found during analysis

1. ****Frontend socket hook is used twice****:
 - â€¢ 'App.jsx' already creates a socket and passes it into 'Dashboard'.
 - â€¢ 'Dashboard.jsx' currently creates a second socket and doesn't use the prop.
 - â€¢ This can prevent live refresh from working as intended.
2. ****Backend DB connection is hard-coded in 'backend/app/db/knex.js'****:
 - â€¢ Docker Compose works because service names ('postgres') are resolvable inside the Docker network.
 - â€¢ If you want to run backend outside Docker, you'll likely want to use env vars consistently (the repo also has 'knexfile.js' that already reads env vars).
3. ****Prisma config exists but doesn't appear to be used****:
 - â€¢ 'backend/app/prisma.config.js' references Prisma, but the app uses Knex migrations for tables.

11) If you want, I can also

- â€¢ Fix the frontend socket/dashboard wiring so that Live updates always work.
- â€¢ Add a Stats row to the dashboard using 'GET /api/stats'.
- â€¢ Add '.env.example' files for frontend and backend.